TAD - Pilha

Prof. Henrique Batista da Silva

Referências

Material adaptado do livro:

ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Java e C++. São Paulo: Thomson Learning, c2007. 621 p.

Os código presentes nos slides estão disponíveis no site do autor (link no último slide)

Pilhas

- Existem aplicações para listas lineares nas quais inserções, retiradas e acessos aos itens ocorrem em um dos extremos da lista.

- Pilha é uma lista linear em que todas as inserções, retiradas e, geralmente, todos os acessos são feitos em apenas um extremo da lista.

Pilhas

- Os itens são colocados um sobre o outro. O item inserido mais recentemente está no topo e o inserido menos recentemente no fundo.

- O modelo intuitivo é o de um monte de pratos em uma prateleira:
  - É conveniente retirar ou adicionar pratos na parte superior.

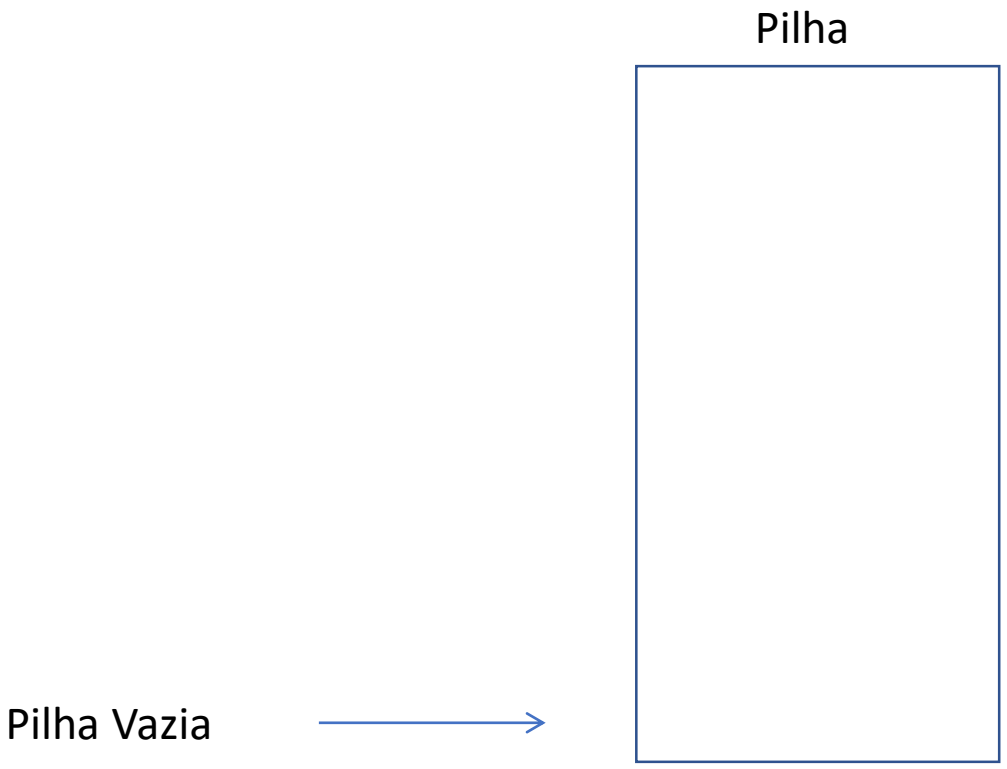
Propriedade das Pilhas

- O último item inserido é o primeiro item que pode ser retirado da lista.
  - Chamadas listas **lifo** (“last-in, first-out”).
- Existe uma ordem linear para pilhas, do “mais recente para o menos recente”.
- É ideal para processamento de estruturas aninhadas de profundidade imprevisível.
  - A ordem de remoção garante que as estruturas mais internas serão processadas antes das mais externas.

Propriedade das Pilhas

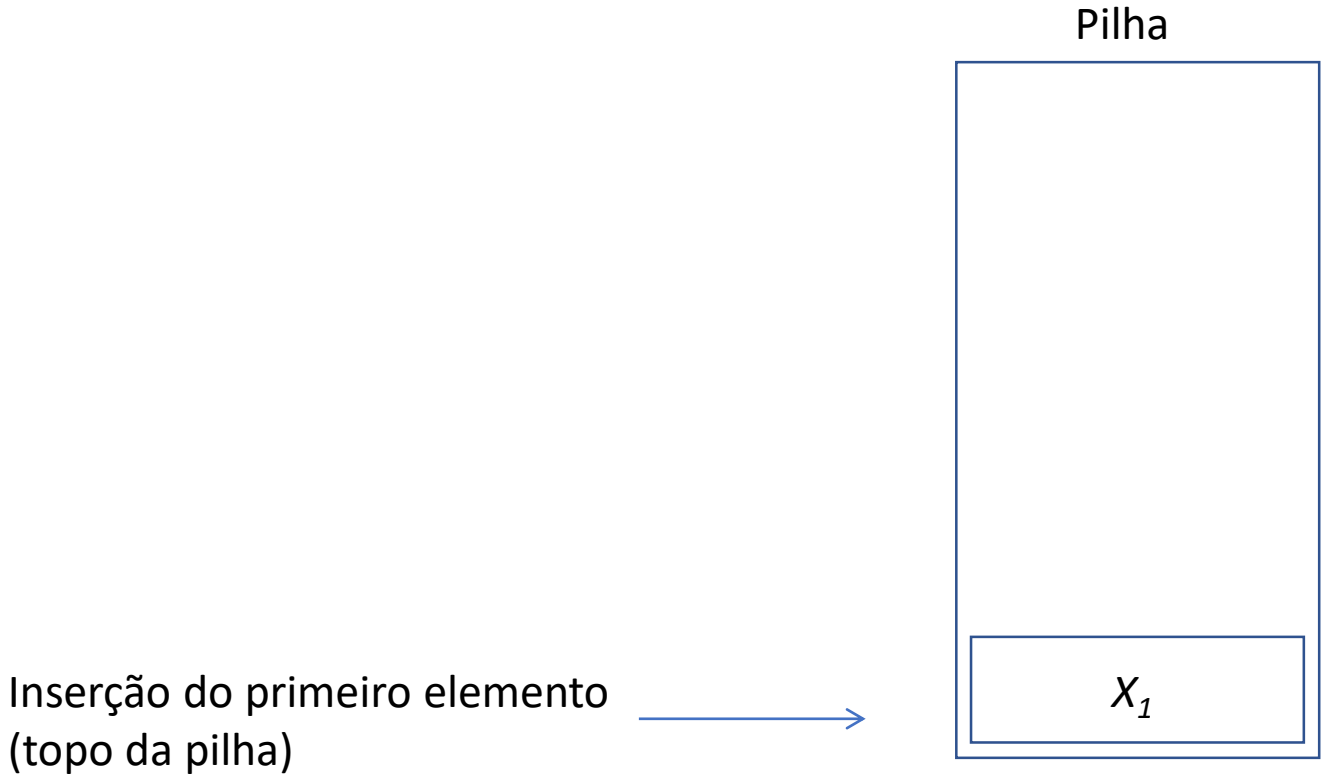
- Aplicações em estruturas aninhadas:
  - Quando é necessário caminhar em um conjunto de dados e guardar uma lista de coisas a fazer posteriormente.
    - O controle de sequências de chamadas de subprogramas.
    - A sintaxe de expressões aritméticas.
- São utilizadas para implementar a recursividade.

Propriedade das Pilhas



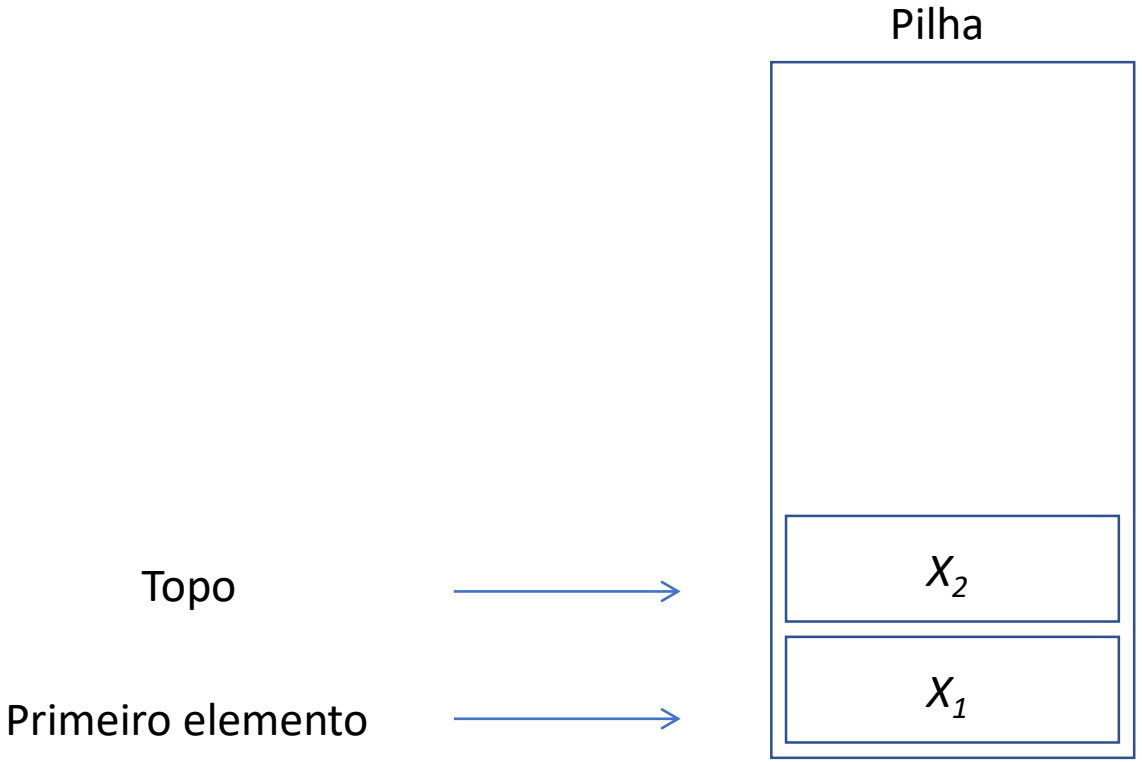
Propriedade das Pilhas

- Inserção de elementos da Pilha



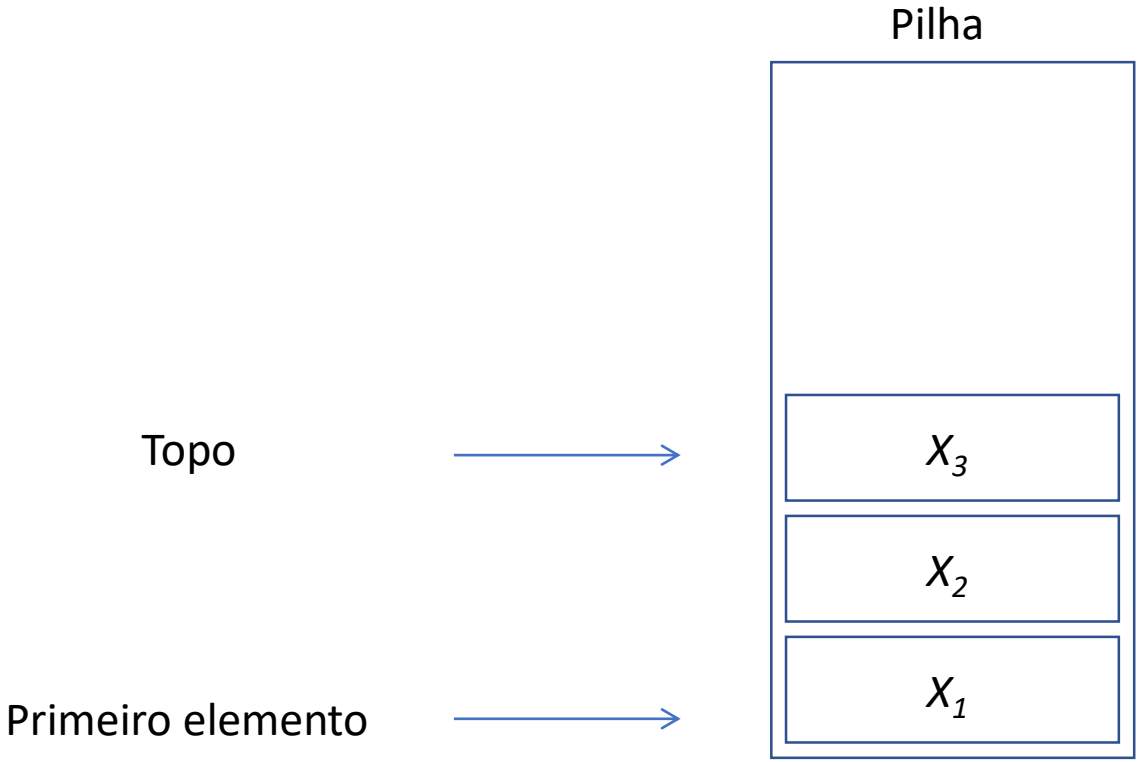
Propriedade das Pilhas

- Inserção de elementos da Pilha



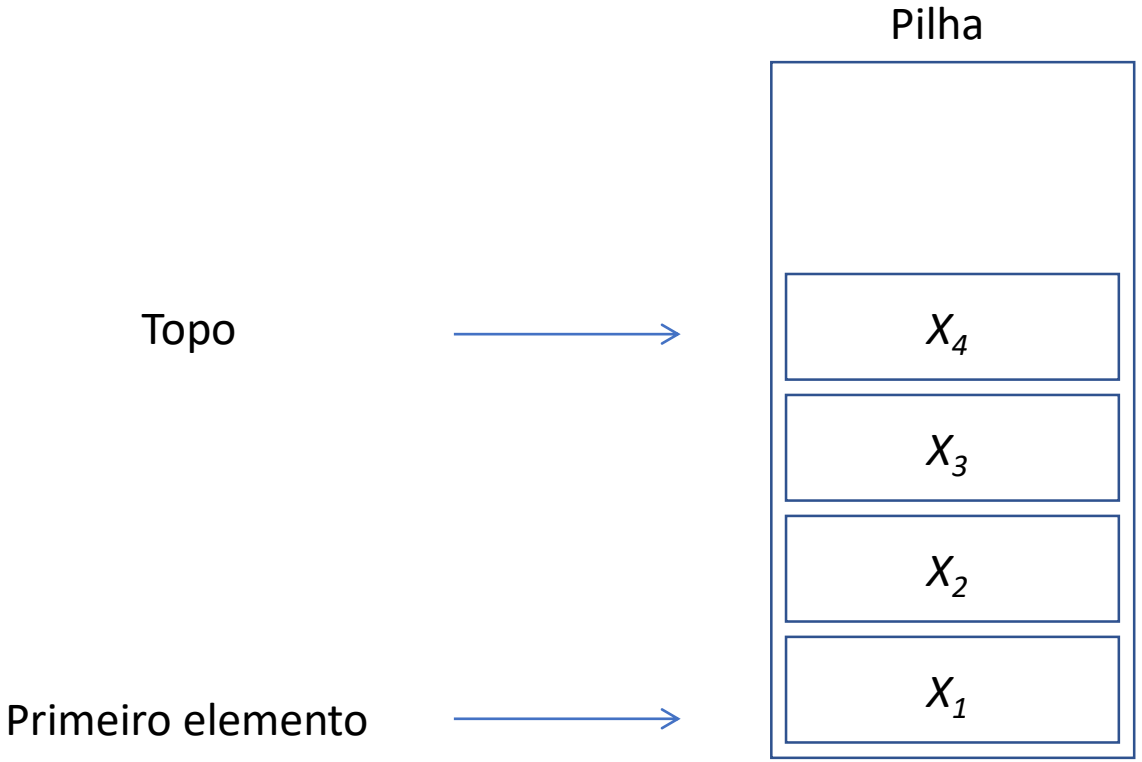
Propriedade das Pilhas

- Inserção de elementos da Pilha



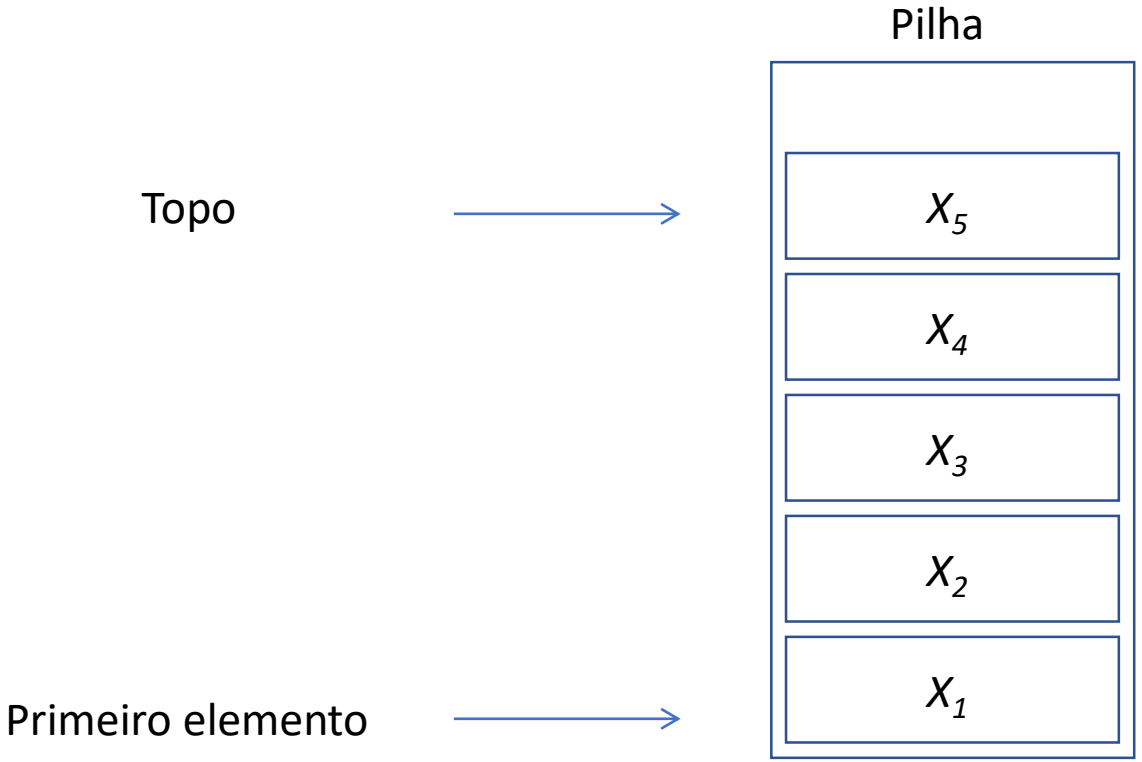
Propriedade das Pilhas

- Inserção de elementos da Pilha



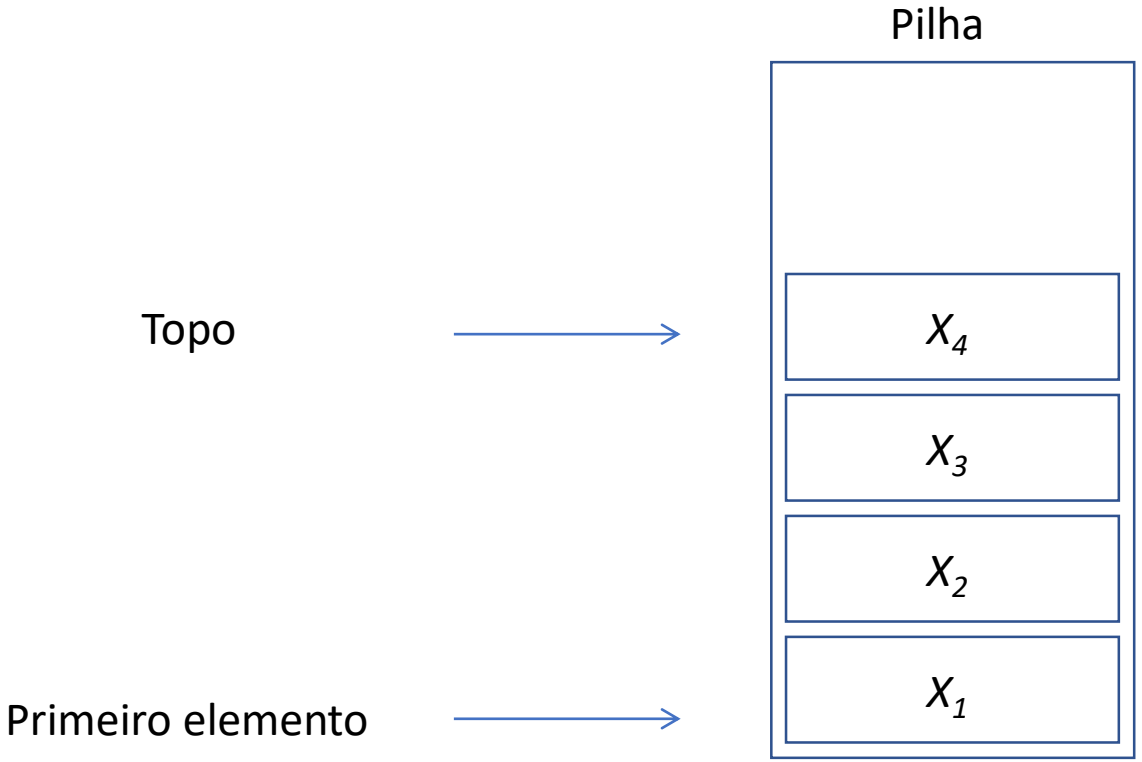
Propriedade das Pilhas

- Inserção de elementos da Pilha



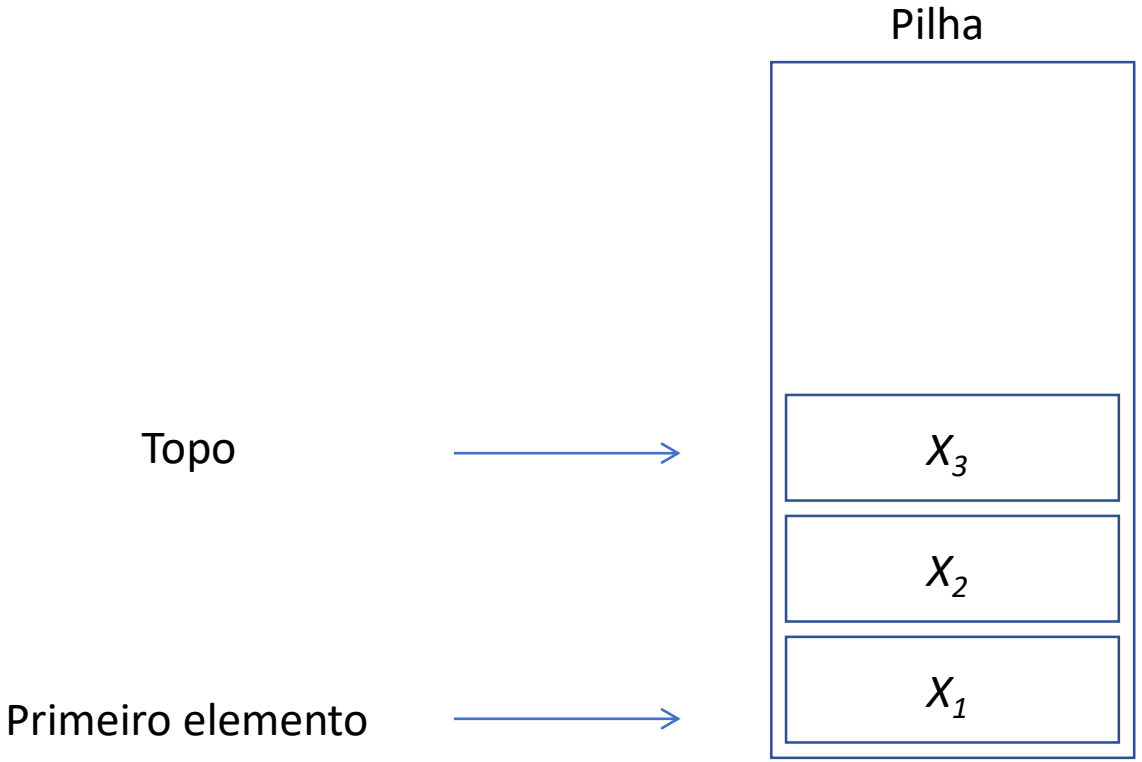
Propriedade das Pilhas

- Remoção de elementos da Pilha



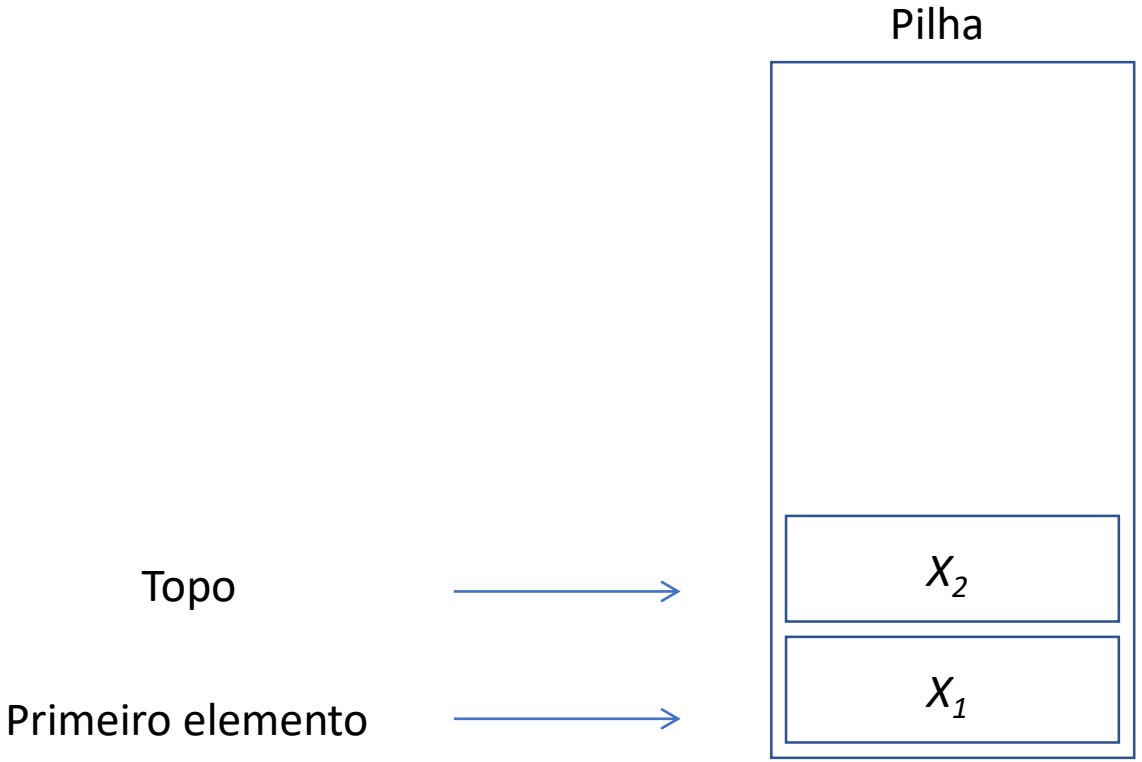
Propriedade das Pilhas

- Remoção de elementos da Pilha



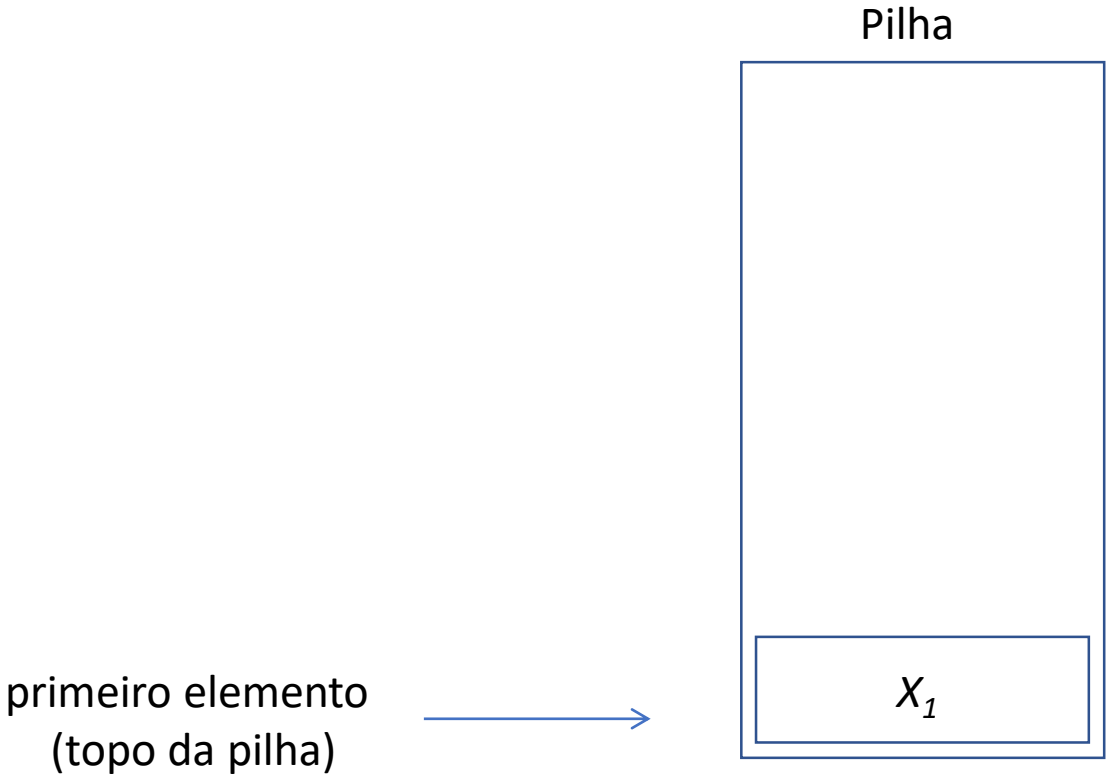
Propriedade das Pilhas

- Remoção de elementos da Pilha



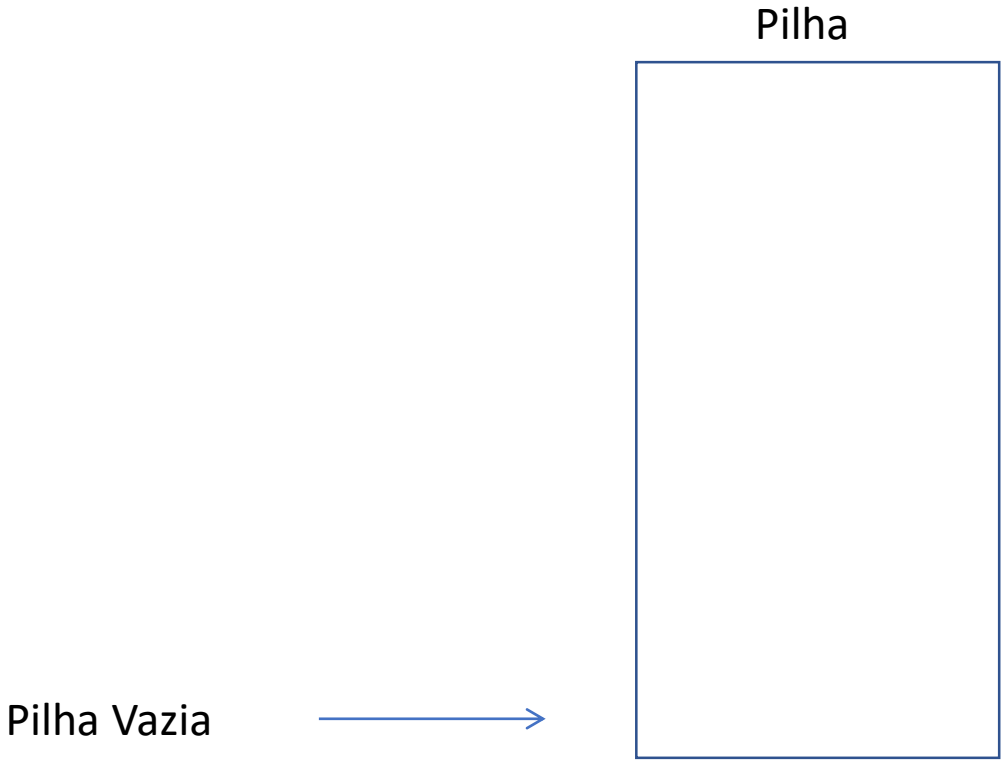
Propriedade das Pilhas

- Remoção de elementos da Pilha



Propriedade das Pilhas

- Remoção de elementos da Pilha



TAD Pilhas

- Conjunto de operações:
  - Cria uma pilha Vazia.
  - Verifica se a lista está vazia. Retorna *true* se a pilha está vazia; caso contrário, retorna *false*.
  - Empilhar o item x no topo da pilha.
  - Desempilhar o item x no topo da pilha, retirando-o da pilha.
  - Verificar o tamanho atual da pilha.

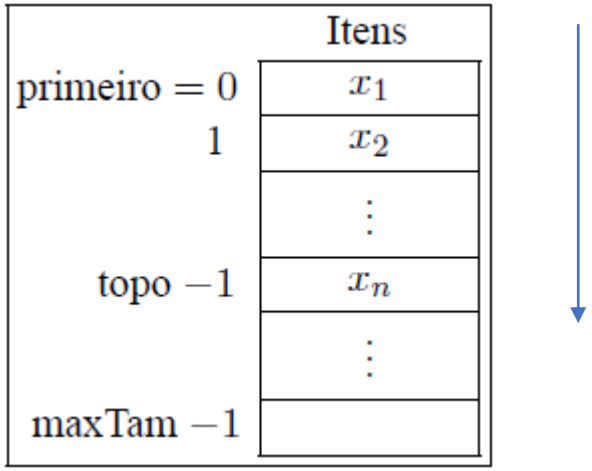
TAD Pilhas

- Existem várias opções de estruturas de dados que podem ser usadas para representar pilhas.
- Representações mais utilizadas:
  - Por meio de *arranjos e*
  - *Estruturas auto referenciadas* (listas ligadas ou encadeadas).

Implementação de Pilhas por meio de Arranjos

- Os itens da pilha são armazenados em posições contíguas de memória
- Inserções e as retiradas ocorrem no topo da pilha:

  - Uso de um cursor chamado **Topo** para controlar a posição do item no topo da pilha.



Direção de inserção da pilha da figura ao lado

Referência: ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Java e C++. São Paulo: Thomson Learning, c2007.

Implementação de Pilhas por meio de Arranjos

- Itens são armazenados em um arranjo de tamanho suficiente para conter a pilha.

- O outro campo do mesmo registro contém uma referência para o item no topo da pilha

- Uma constante define o **tamanho máximo** permitido para a pilha

Implementação de Pilhas por meio de Arranjos

```
public class Pilha {
  private Object item[];
  private int    topo;

  public Pilha () {
    this.item = new Object[1000]; this.topo = 0;
  }

  public Pilha (int maxTam) {
    this.item = new Object[maxTam]; this.topo = 0;
  }

  public void empilha (Object x) throws Exception {
    if (this.topo == this.item.length)
      throw new Exception ("Erro: A pilha esta cheia");
    else this.item[this.topo++] = x;
  }
```

Estrutura (Vetor) do tipo Object (tipo genérico em Java)

Variável que controla (armazena) a posição do elemento do topo da pilha

Construtor: cria uma pilha vazia com tamanho máximo definido. Observe que quando a pilha está vazia “topo” é igual a zero.

Construtor: cria uma pilha vazia com tamanho máximo definido pelo usuário

Método empilha: empilha novo item se houver espaço na pilha. Observe que a cada novo elemento inserido, “topo” é incrementado.

Referência: ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Java e C++. São Paulo: Thomson Learning, c2007.

Implementação de Pilhas por meio de Arranjos

```
public Object desempilha () throws Exception {
    if (this.vazia())
      throw new Exception ("Erro: A pilha esta vazia");
    return this.item[--this.topo];
  }

  public boolean vazia () {
    return (this.topo == 0);
  }

  public int tamanho () {
    return this.topo;
  }
}
```

Método desempilha: desempilha o item do topo se a pilha não estiver vazia (quando topo é igual a zero). Observe que a cada elemento removido, “topo” é decrementado.

Retorna *true* se a pilha estiver vazia

Retorna *a quantidade* de elementos da pilha.

Referência: ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Java e C++. São Paulo: Thomson Learning, c2007.

Implementação de Pilhas por meio de Estruturas Auto Referenciadas

Implementação de Pilhas por meio de Estruturas Auto Referenciadas

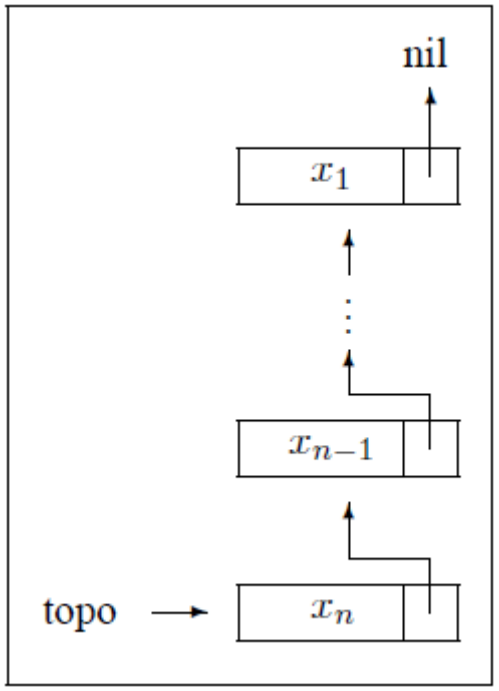
- Ao contrário da implementação de listas lineares, por meio e estruturas auto referenciadas não há necessidade de manter uma célula *head*:
  - Mantem-se o controle pelo topo da pilha.

- Para desempilhar um item, basta “desconectar” a célula que contém x_n e a célula que contém x_{n-1} passa a ser a célula de topo.

- Não há mais controle sobre o tamanho máximo da pilha, que pode crescer enquanto houver memória disponível no sistema.

Implementação de Pilhas por meio de Estruturas Auto Referenciadas

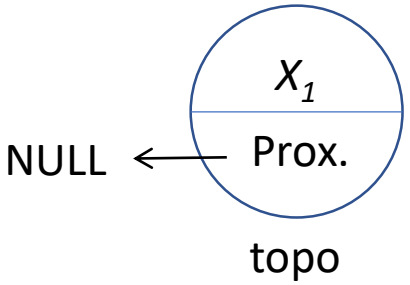
- Para empilhar um novo item, basta fazer a operação contrária, criando uma nova célula para receber o novo item.

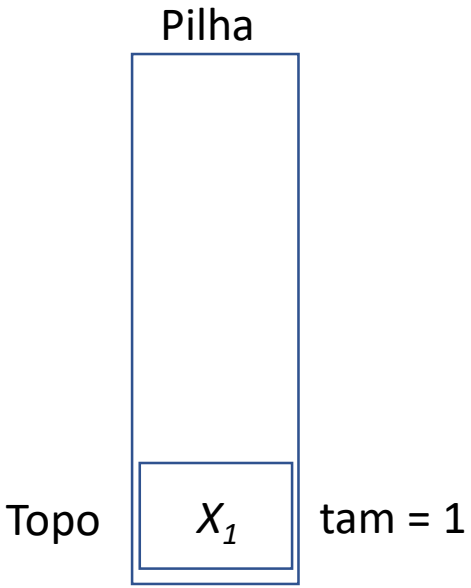


Referência: ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Java e C++. São Paulo: Thomson Learning, c2007.

Implementação de Pilhas por meio de Estruturas Auto Referenciadas

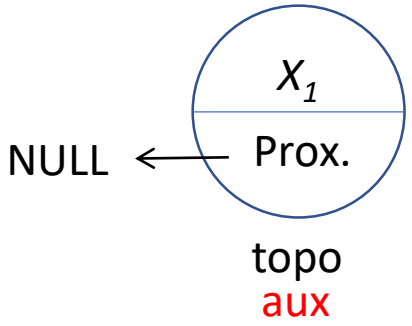
- Pilha com apenas um elemento:

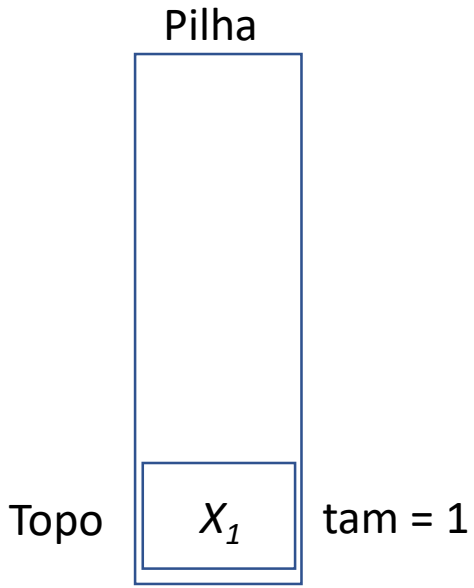




Implementação de Pilhas por meio de Estruturas Auto Referenciadas

- Inserção de novos elementos na pilha (empilha):

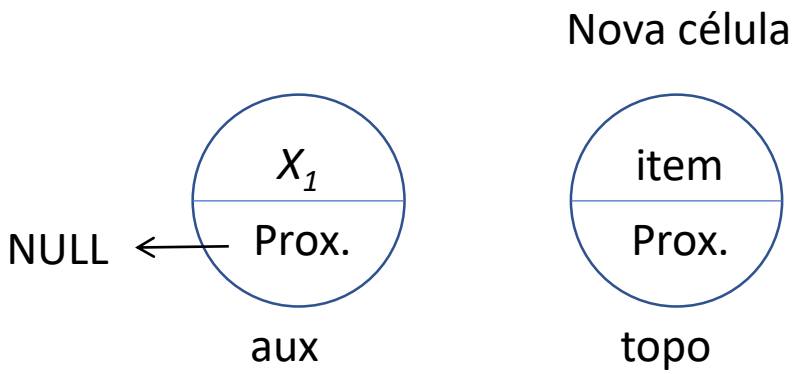


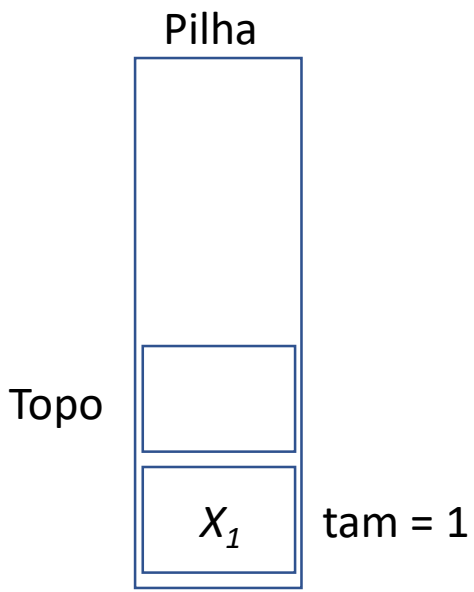



```
public void empilha (Object x) {
   Celula aux = this.topo;
   this.topo = new Celula ();
   this.topo.item = x;
   this.topo.prox = aux;
   this.tam++;
 }
```

Implementação de Pilhas por meio de Estruturas Auto Referenciadas

- Inserção de novos elementos na pilha (empilha):

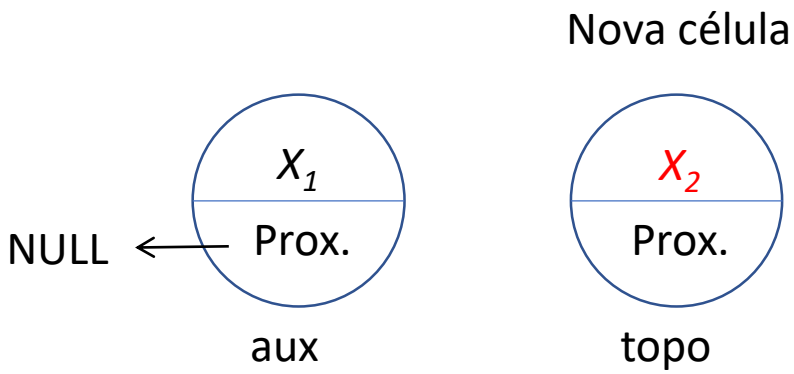


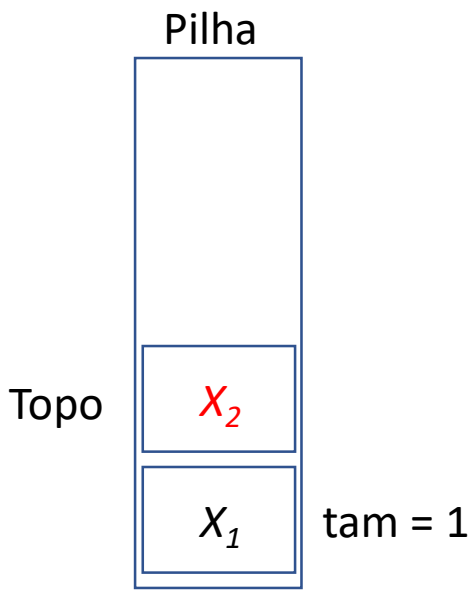



```
public void empilha (Object x) {
   Celula aux = this.topo;
   this.topo = new Celula ();
   this.topo.item = x;
   this.topo.prox = aux;
   this.tam++;
 }
```

Implementação de Pilhas por meio de Estruturas Auto Referenciadas

- Inserção de novos elementos na pilha (empilha):

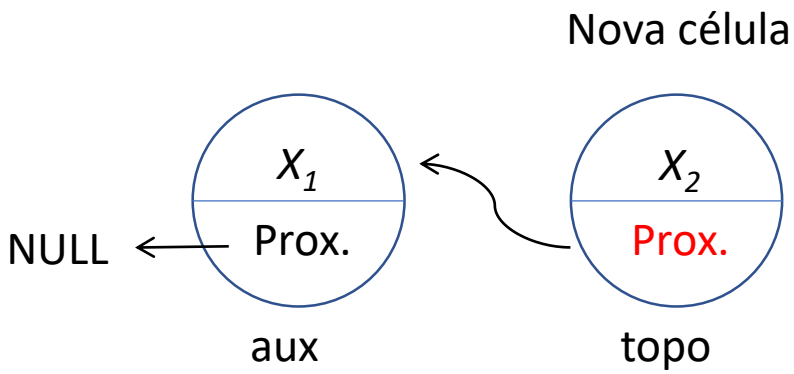


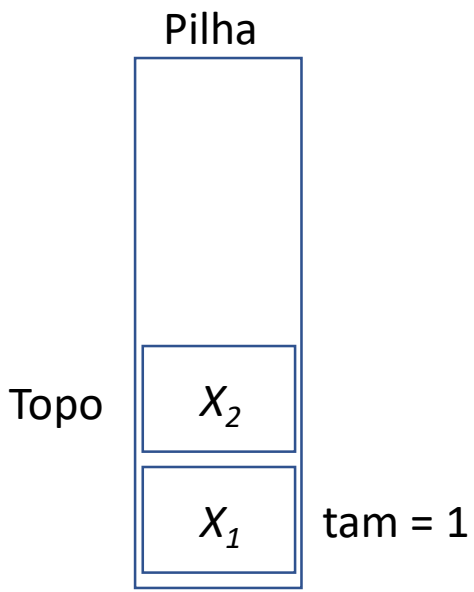



```
public void empilha (Object x) {
   Celula aux = this.topo;
   this.topo = new Celula ();
   this.topo.item = x;
   this.topo.prox = aux;
   this.tam++;
 }
```

Implementação de Pilhas por meio de Estruturas Auto Referenciadas

- Inserção de novos elementos na pilha (empilha):

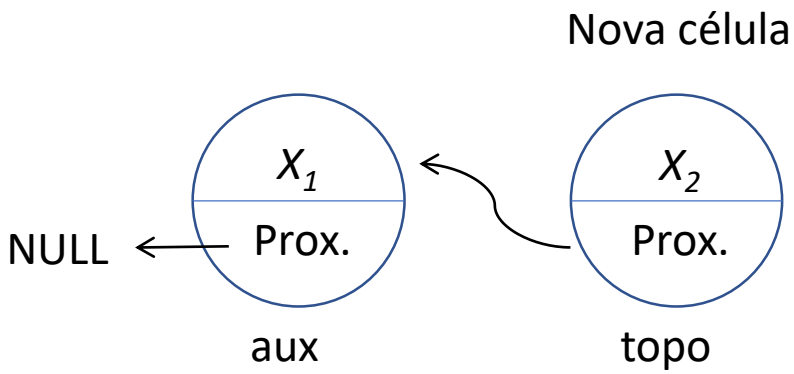


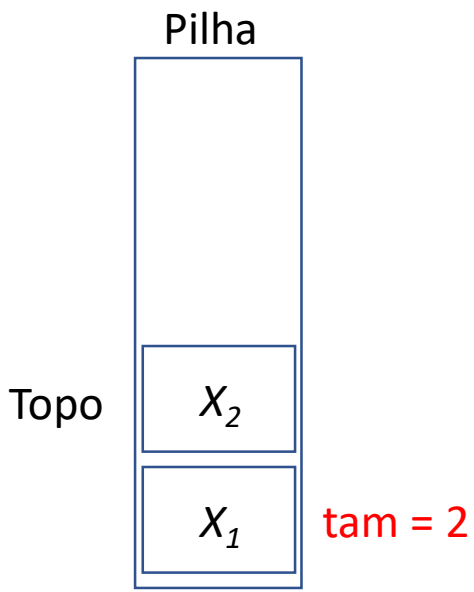



```
public void empilha (Object x) {
   Celula aux = this.topo;
   this.topo = new Celula ();
   this.topo.item = x;
   this.topo.prox = aux;
   this.tam++;
 }
```

Implementação de Pilhas por meio de Estruturas Auto Referenciadas

- Inserção de novos elementos na pilha (empilha):

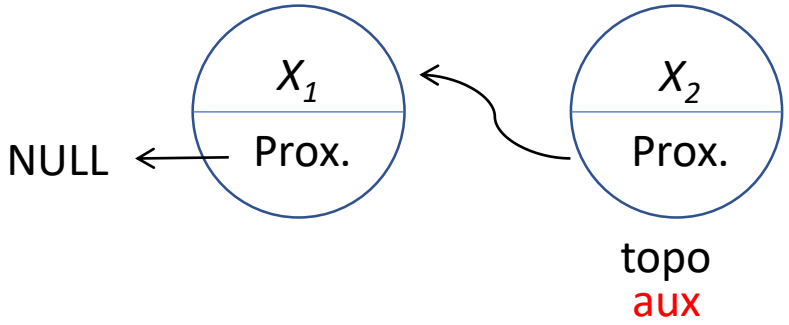


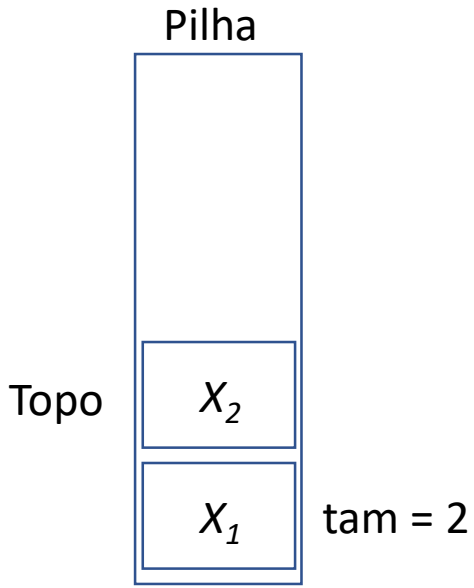



```
public void empilha (Object x) {
   Celula aux = this.topo;
   this.topo = new Celula ();
   this.topo.item = x;
   this.topo.prox = aux;
   this.tam++;
 }
```

Implementação de Pilhas por meio de Estruturas Auto Referenciadas

- Inserção de novos elementos na pilha (empilha):

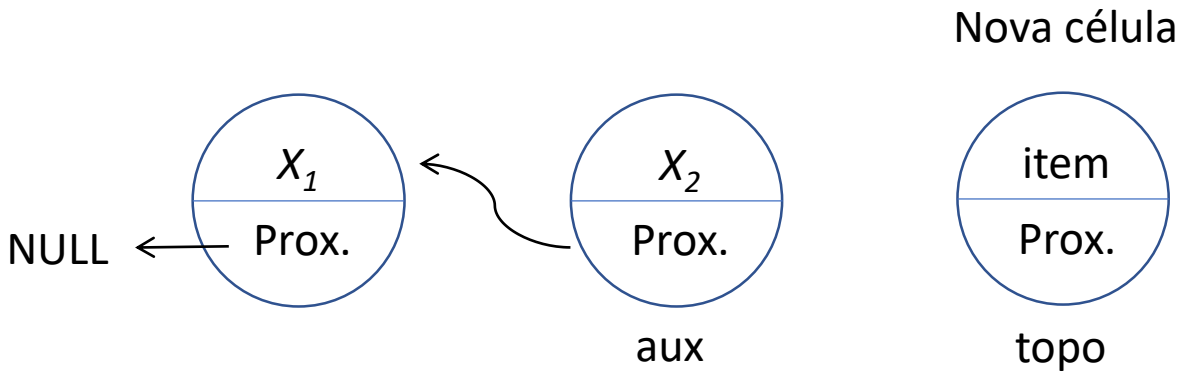


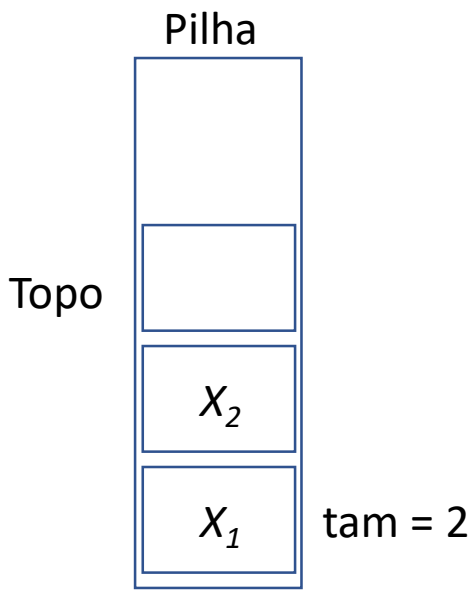



```
public void empilha (Object x) {
   Celula aux = this.topo;
   this.topo = new Celula ();
   this.topo.item = x;
   this.topo.prox = aux;
   this.tam++;
 }
```

Implementação de Pilhas por meio de Estruturas Auto Referenciadas

- Inserção de novos elementos na pilha (empilha):

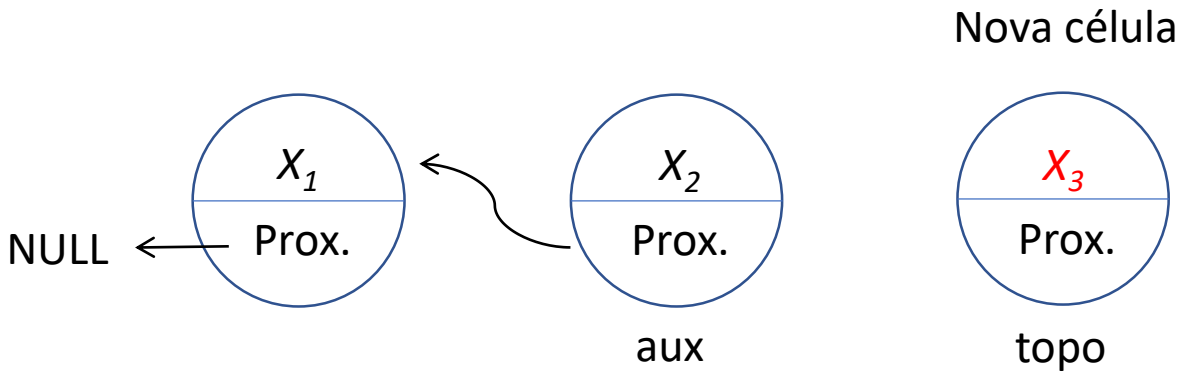


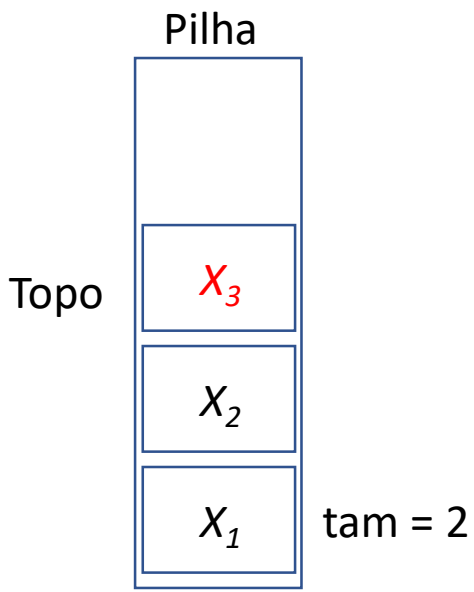



```
public void empilha (Object x) {
   Celula aux = this.topo;
   this.topo = new Celula ();
   this.topo.item = x;
   this.topo.prox = aux;
   this.tam++;
 }
```

Implementação de Pilhas por meio de Estruturas Auto Referenciadas

- Inserção de novos elementos na pilha (empilha):

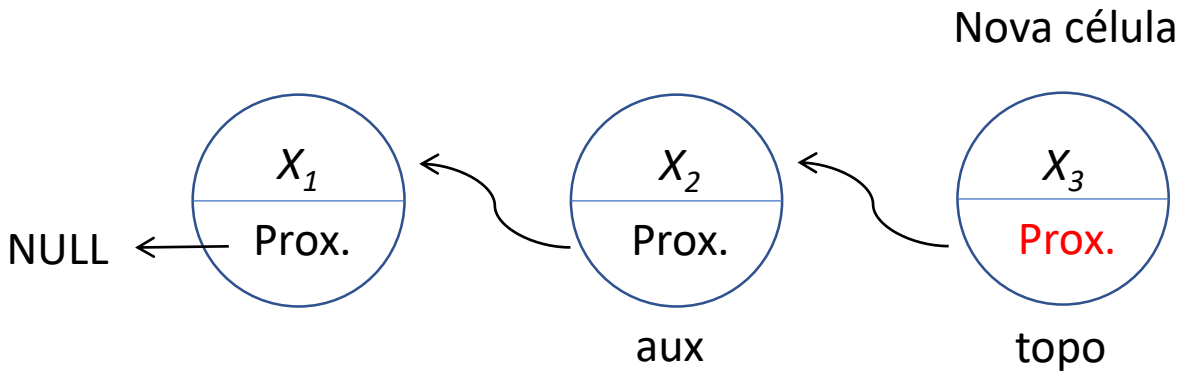


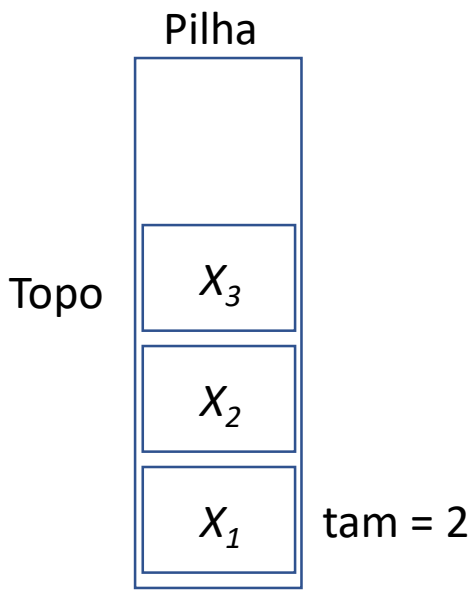



```
public void empilha (Object x) {
   Celula aux = this.topo;
   this.topo = new Celula ();
   this.topo.item = x;
   this.topo.prox = aux;
   this.tam++;
 }
```

Implementação de Pilhas por meio de Estruturas Auto Referenciadas

- Inserção de novos elementos na pilha (empilha):

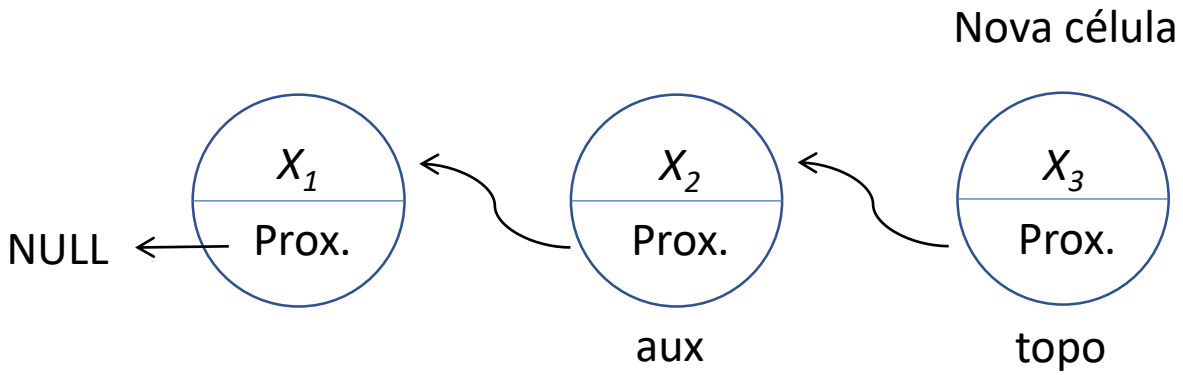


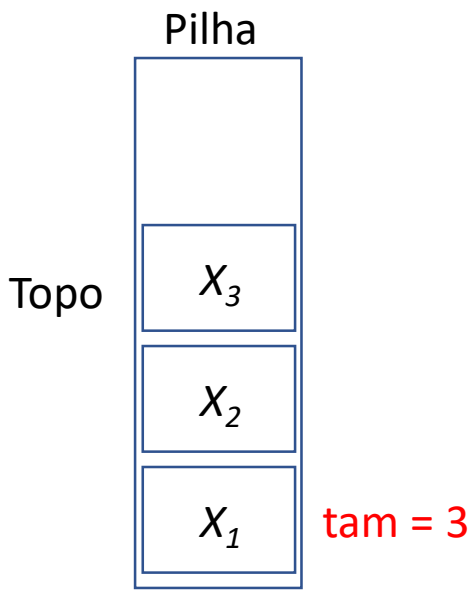



```
public void empilha (Object x) {
   Celula aux = this.topo;
   this.topo = new Celula ();
   this.topo.item = x;
   this.topo.prox = aux;
   this.tam++;
 }
```

Implementação de Pilhas por meio de Estruturas Auto Referenciadas

- Inserção de novos elementos na pilha (empilha):

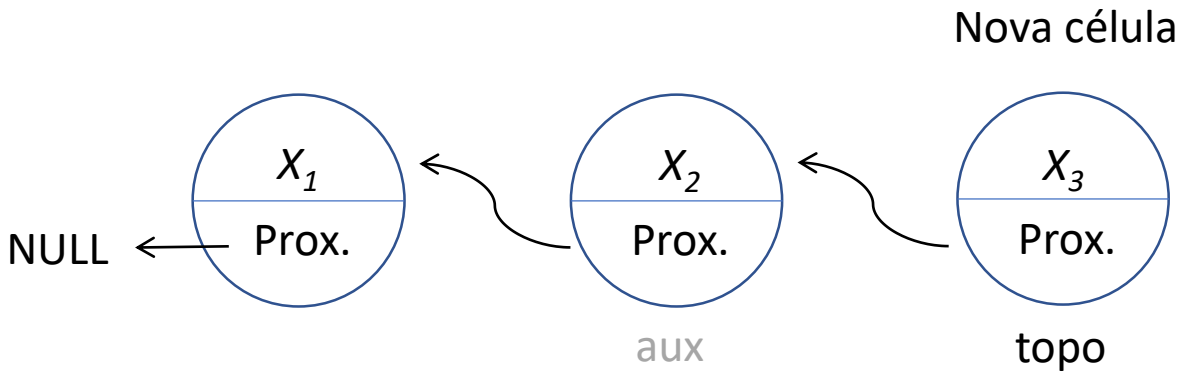


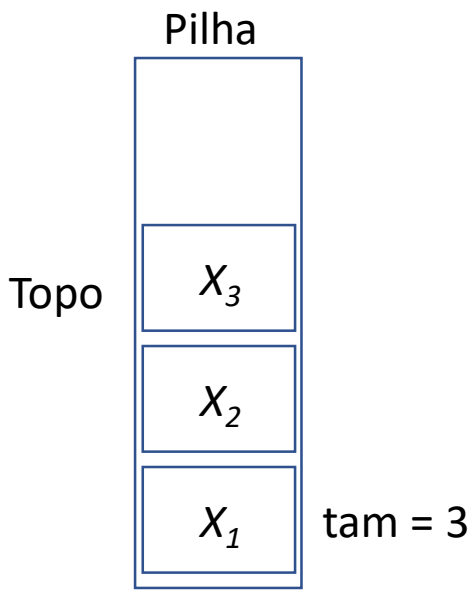



```
public void empilha (Object x) {
   Celula aux = this.topo;
   this.topo = new Celula ();
   this.topo.item = x;
   this.topo.prox = aux;
   this.tam++;
 }
```

Implementação de Pilhas por meio de Estruturas Auto Referenciadas

- Inserção de novos elementos na pilha (empilha):



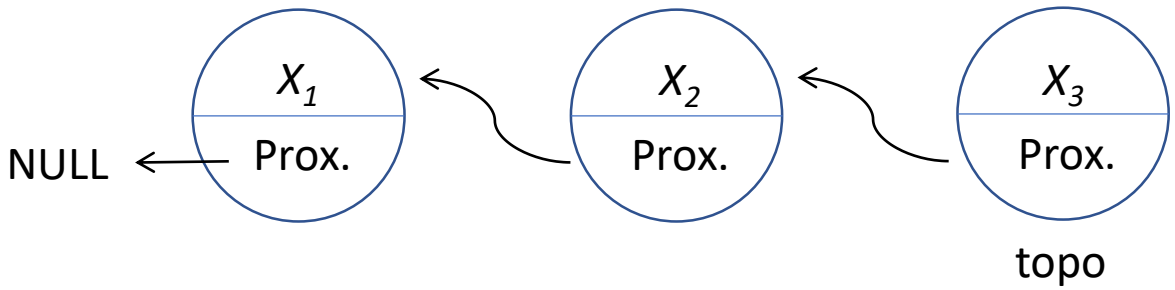


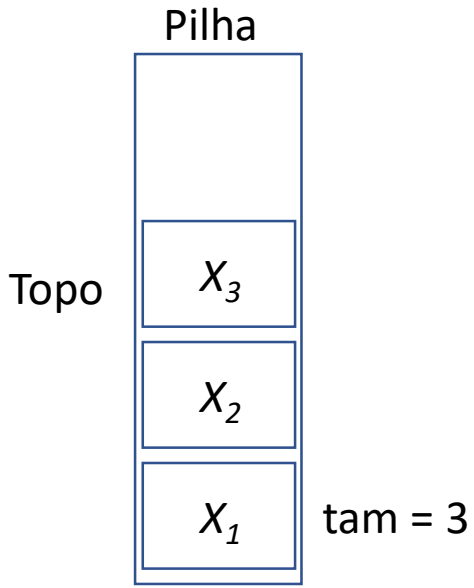
Ao final do método *empilha*, "aux" deixa de existir fora do escopo da função.

```
public void empilha (Object x) {
    Celula aux = this.topo;
    this.topo = new Celula ();
    this.topo.item = x;
    this.topo.prox = aux;
    this.tam++;
}
```

Implementação de Pilhas por meio de Estruturas Auto Referenciadas

- Remoção de itens na pilha (desempilha):

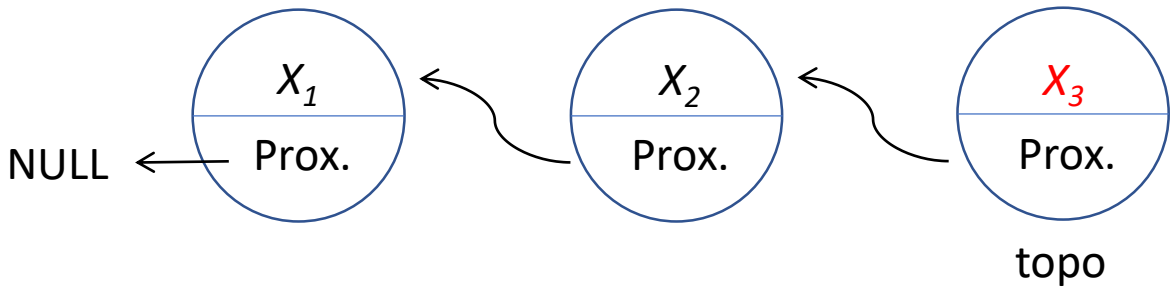


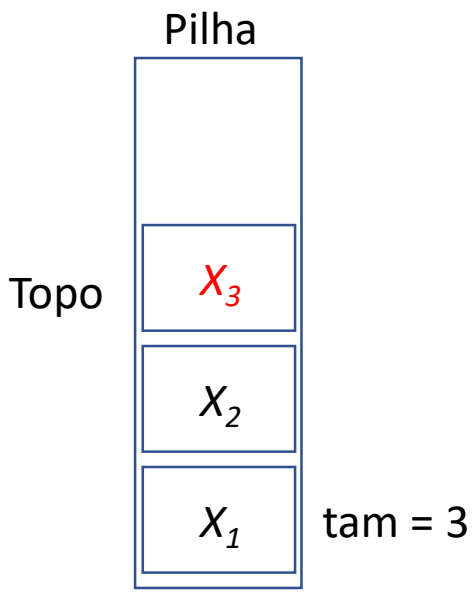



```
public Object desempilha () throws Exception {
    if (this.vazia ())
      throw new Exception ("Erro: A pilha esta vazia");
    Object item = this.topo.item;
    this.topo = this.topo.prox; this.tam--;
    return item;
}
```

Implementação de Pilhas por meio de Estruturas Auto Referenciadas

- Remoção de itens na pilha (desempilha):

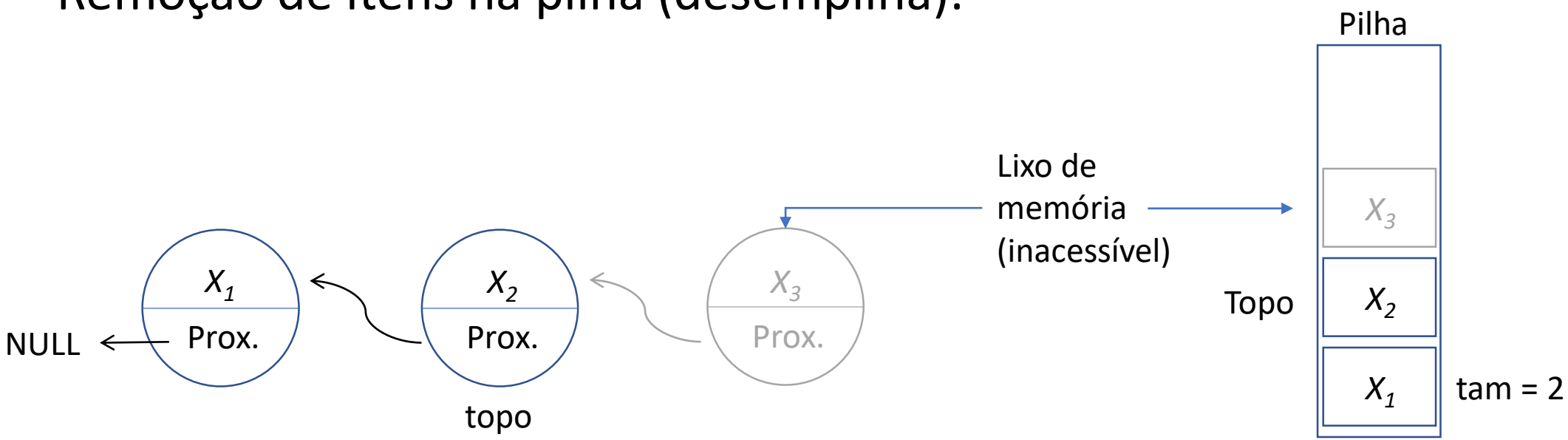





```
public Object desempilha () throws Exception {
    if (this.vazia ())
      throw new Exception ("Erro: A pilha esta vazia");
    Object item = this.topo.item;
    this.topo = this.topo.prox; this.tam--;
    return item;
}
```

Implementação de Pilhas por meio de Estruturas Auto Referenciadas

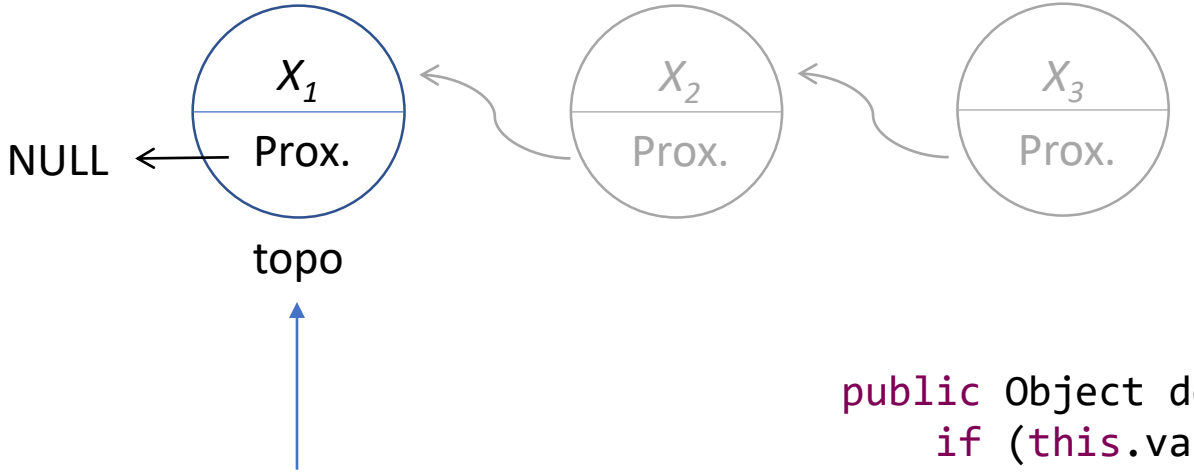
- Remoção de itens na pilha (desempilha):

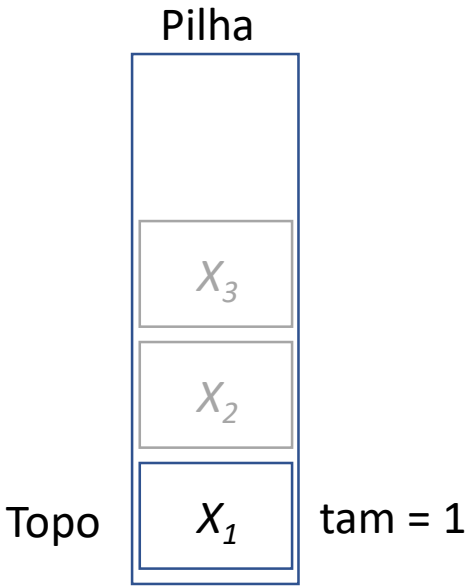



```
public Object desempilha () throws Exception {
    if (this.vazia ())
      throw new Exception ("Erro: A pilha esta vazia");
    Object item = this.topo.item;
    this.topo = this.topo.prox; this.tam--;
    return item;
}
```

Implementação de Pilhas por meio de Estruturas Auto Referenciadas

- Remoção de itens na pilha (desempilha):



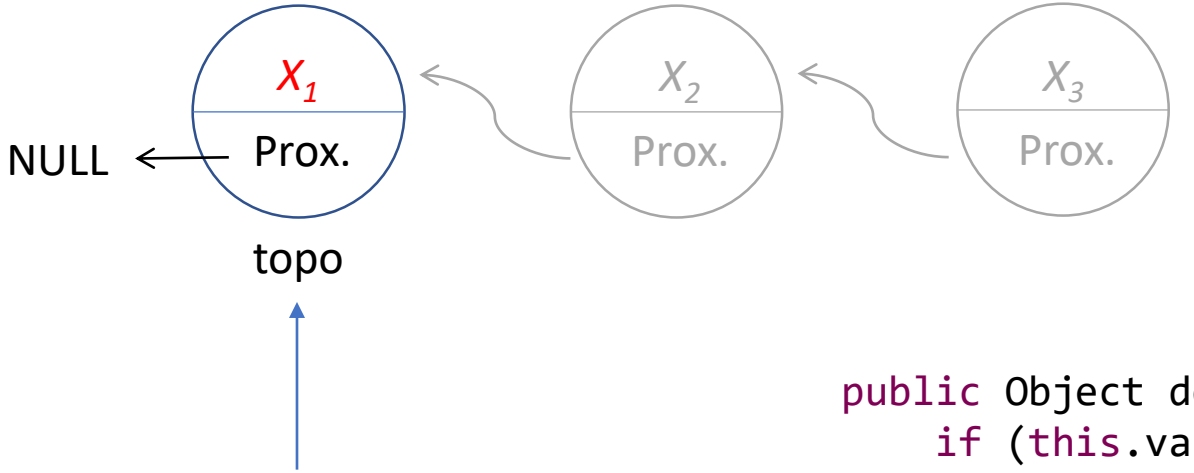


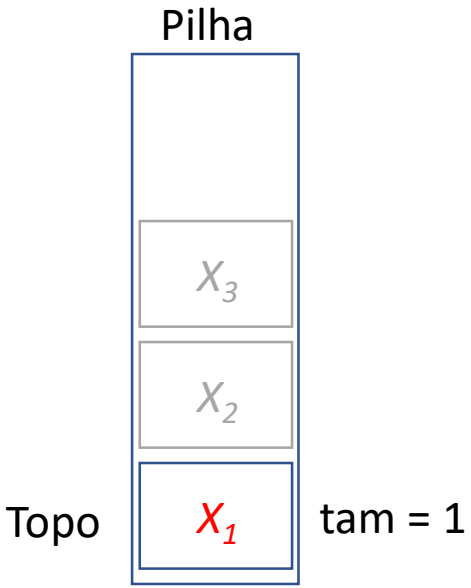
Observe que o primeiro elemento inserido na pilha sempre faz referência para NULL. Quando este elemento for removido, topo passará a referências NULL, caracterizando **pilha vazia**.

```
public Object desempilha () throws Exception {
    if (this.vazia ())
      throw new Exception ("Erro: A pilha esta vazia");
    Object item = this.topo.item;
    this.topo = this.topo.prox; this.tam--;
    return item;
}
```

Implementação de Pilhas por meio de Estruturas Auto Referenciadas

- Remoção de itens na pilha (desempilha):



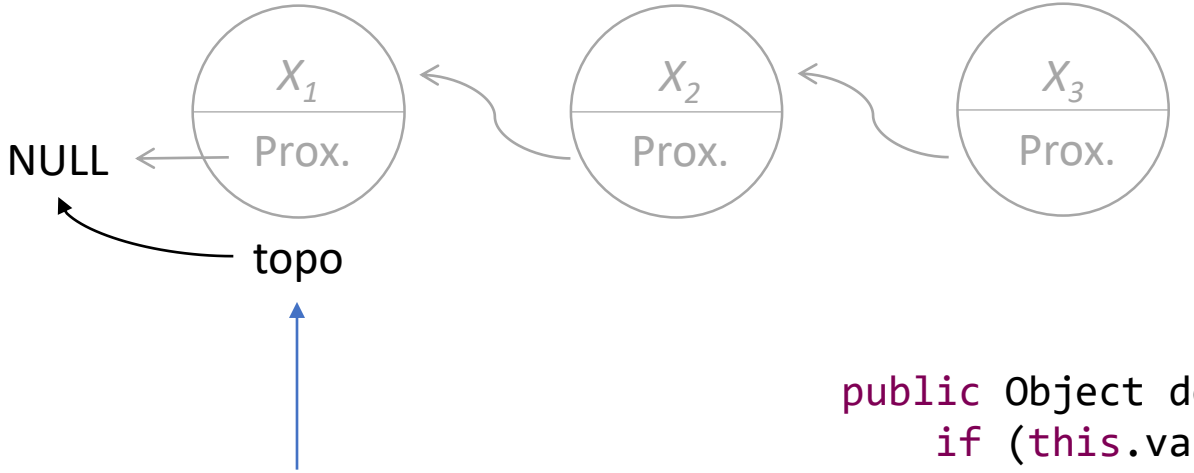


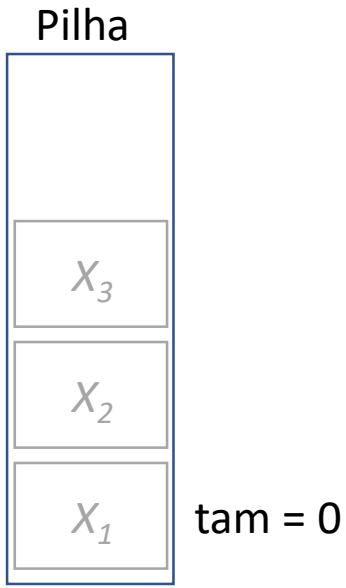
Observe que o primeiro elemento inserido na pilha sempre faz referência para NULL. Quando este elemento for removido, topo passará a referências NULL, caracterizando **pilha vazia**.

```
public Object desempilha () throws Exception {
    if (this.vazia ())
      throw new Exception ("Erro: A pilha esta vazia");
    Object item = this.topo.item;
    this.topo = this.topo.prox; this.tam--;
    return item;
}
```

Implementação de Pilhas por meio de Estruturas Auto Referenciadas

- Remoção de itens na pilha (desempilha):





Observe que o primeiro elemento inserido na pilha sempre faz referência para NULL. Quando este elemento for removido, topo passará a referências NULL, caracterizando **pilha vazia**.

```
public Object desempilha () throws Exception {
    if (this.vazia ())
      throw new Exception ("Erro: A pilha esta vazia");
    Object item = this.topo.item;
    this.topo = this.topo.prox; this.tam--;
    return item;
}
```

Operações sobre Pilhas Usando Estruturas Auto Referenciadas

- O campo *tam* evita a contagem do número de itens no método *tamanho*.
- Cada célula de uma pilha contém um item da pilha e uma referência para outra célula.
- A classe Pilha contém uma referência para o topo da pilha.

Pilhas Usando Estruturas Auto Referenciadas

```
public class Pilha {
  private static class Celula {
    Object item;
    Celula prox;
  }
  private Celula topo;
  private int    tam;

  public Pilha () {
    this.topo = null; this.tam = 0;
  }
  public void empilha (Object x) {
    Celula aux = this.topo;
    this.topo = new Celula ();
    this.topo.item = x;
    this.topo.prox = aux;
    this.tam++;
  }
```

Observe que a classe pilha contém a classe “Celula”. Esta classe possui um objeto item (tipo genérico) e um objeto “prox” que é uma referência para a própria classe “Celula”.

“topo” é do tipo “Celula”, pois sempre irá armazenar a célula do elemento do topo da pilha.

Criando uma pilha vazia. “topo” é um objeto não instanciado e portanto faz referência para NULL.

Referência: ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Java e C++. São Paulo: Thomson Learning, c2007.

Pilhas Usando Estruturas Auto Referenciadas

```
 public Object desempilha () throws Exception {
    if (this.vazia ())
      throw new Exception ("Erro: A pilha esta vazia");
    Object item = this.topo.item;
    this.topo = this.topo.prox;
    this.tam--;
    return item;
  }
  public boolean vazia () {
    return (this.topo == null);
  }
  public int tamanho () {
    return this.tam;
  }
}
```


Referência: ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Java e C++. São Paulo: Thomson Learning, c2007.

Teste da pilha

```
public class TestaPilha {
  public static void main (String[] args) {
    Pilha pilha = new Pilha ();
    Integer vetor[] = new Integer[1000];
    Integer n = null;
    int i, j, k, max;
    java.util.Random random = new java.util.Random ();
    max = 10;
    try {
      // Gera uma permutacao aleatoria de chaves entre 0 e max - 1
      for (i = 0; i < max; i++)
        vetor[i] = new Integer (i);
      for (i = 0; i < max; i++) {
        k = Math.abs (random.nextInt ()) % max;
        j = Math.abs (random.nextInt ()) % max;
        n = vetor[k];
        vetor[k] = vetor[j];
        vetor[j] = n;
      }
```

Utilize a classe ao lado para testar a pilha por vetor e com estruturas auto referenciadas

Referência: ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Java e C++. São Paulo: Thomson Learning, c2007.

Teste da pilha

```
      // Insere cada chave na pilha
      for (i = 0; i < max; i++) {
        pilha.empilha (vetor[i]);
        System.out.println ("Empilhou: " + vetor[i].toString ());
      }
      System.out.println ("Tamanho da pilha: " + pilha.tamanho ());
      // Desempilha cada chave
      for (i = 0; i < max; i++) {
        n = (Integer) pilha.desempilha ();
        System.out.println ("Desempilhou: " + n.toString ());
      }
      n = (Integer) pilha.desempilha ();
    } catch (Exception e) {
      System.out.println (e.getMessage ());
    }
    System.out.println ("Tamanho da pilha: " + pilha.tamanho ());
  }
}
```


Referência: ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Java e C++. São Paulo: Thomson Learning, c2007.

Referências

- Baseado no material de aulas de:
  - ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Java e C++. São Paulo: Thomson Learning, c2007. 621 p.
  - Para downloads dos códigos em Java:
  - http://www2.dcc.ufmg.br/livros/algoritmos-java/implementacoes-03.php

- Outras referências:
  - CORMEN, Thomas H. et al. Algoritmos: teoria e prática. Rio de Janeiro: Campus, 2002. 916p. ISBN 8535209263 (Disponível no Acervo).

  - DROZDEK, Adam. Estrutura de dados e algoritmos em C++. São Paulo: Pioneira Thomson Learning, 2002. xviii, 579p. ISBN 8522102953 (Disponível no Acervo).